Aula 15

Análise de Algoritmos

Cris Sato

Enumeração e Força Bruta
(Parte 2)

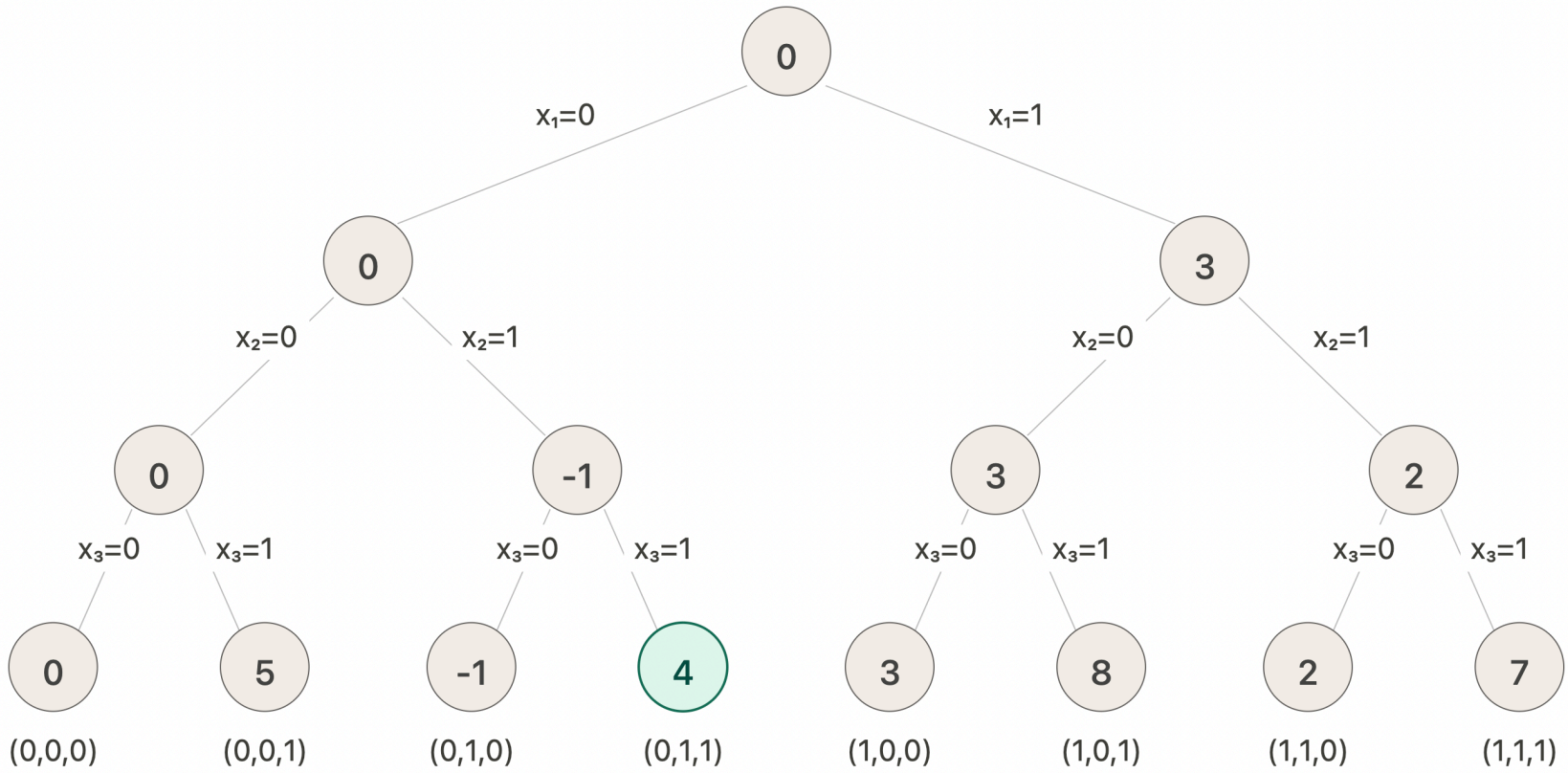
Problema: Número de subconjuntos que somam S
Entrada: um vetor A[1..n] de inteiros (positivos e negativos) e um número inteiro S
Saída: número de tuplas binárias $(x_1, x_2, ..., x_n)$ tal que $x_1A[1] + x_2A[2] + ... + x_nA[n] = S$

- Neste problema, não queremos apenas encontrar uma tupla satisfazendo a condição, queremos contar o número de tais tuplas. A maneira mais direta é por força bruta, pois o número de tuplas pode ser exponencial

Na aula anterior vimos como gerar todos os subconjuntos de tamanho k de {1, ..., n}. Cada subconjunto era associado a uma tupla crescente $(i_1, ..., i_k)$. Naturalmente, poderíamos usar esse algoritmo iterando k de 0 a n. Isso faz sentido se quisermos gerar os conjuntos por ordem de tamanho. Mas o algoritmo fica mais simples sem essa restrição.

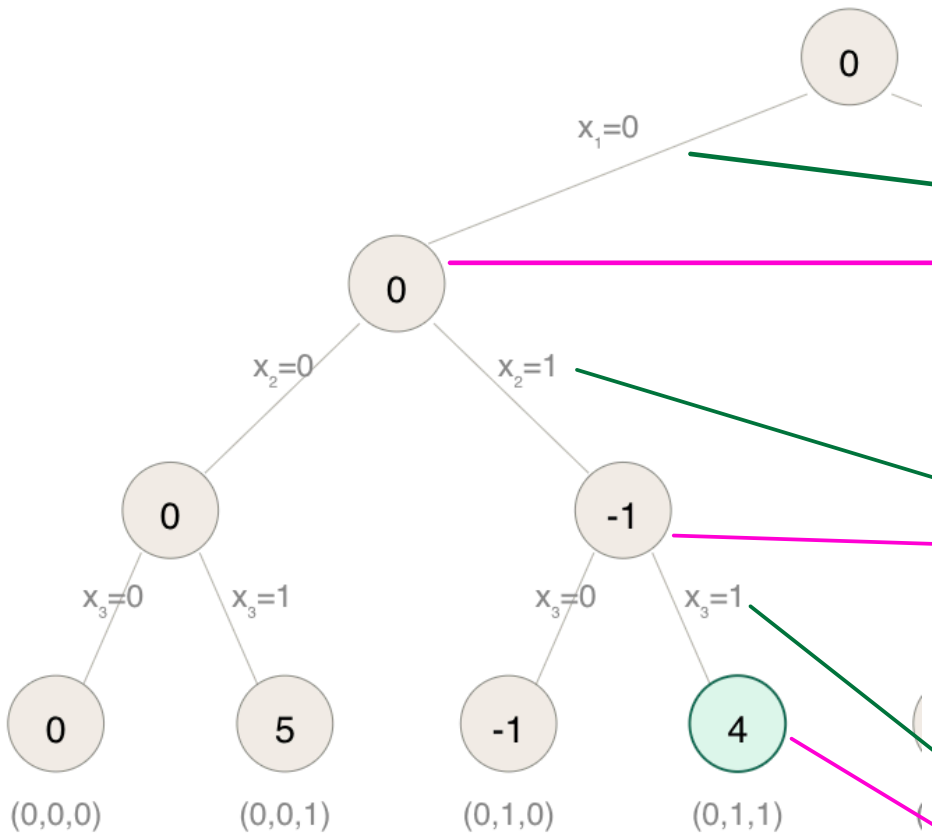
Vamos ver a versão recursiva primeiro. A ideia é gerar recursivamente:

- todos os subconjuntos que incluem 1
- todos os subconjuntos que não incluem 1



A = [3, -1, 5], S = 4

```
incluido[1..n] = vetor binário de n
posições

def subconjuntos(i)
    se i == n + 1:
        imprimir(incluido)
    senão:
        incluido[i] = 0
        subconjuntos(i + 1)
        incluido[i] = 1
        subconjuntos(i + 1)
```



subconjuntos(1)
incluido[1]=0
subconjuntos(2)

subconjuntos(2)
incluido[2]=1
subconjuntos(3)

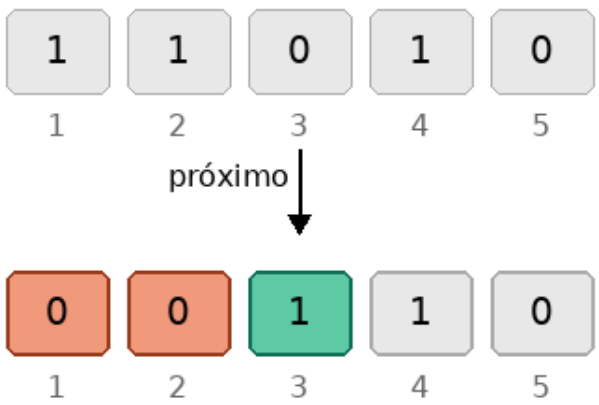
subconjuntos(3)
incluido[1]=1
subconjuntos(4)

- Basta checar a soma a cada conjunto gerado e a soma na chamada recursiva é ajustada quando decidimos incluir A[i]

```
incluido[1..n] = vetor binário de n
posições

def soma(A, S, i)
  se i == n + 1
    se S == 0
      imprimir(incluido)
  senão
    incluido[i] = 0
    soma(A, S, i + 1)
    incluido[i] = 1
    soma(A, S - A[i], i + 1)
```

- Este código faz o que queremos, mas tem um defeito de design: a geração dos subconjuntos está misturada com a checagem da soma. O ideal seria separar essas duas partes, mas sem armazenar todos os subconjuntos, pois isso gastaria espaço exponencial.
- É difícil de conseguir fazer isso na versão recursiva . Na versão iterativa, é bem mais fácil

```
incluido[1..n]=inicializado com 0's

def próximo(n)
  para i = 0 até n
    se incluido[i] == 0
      incluido[i] = 1
      retornar Verdadeiro
    senão
      incluido[i] = 0
  retornar Falso
```

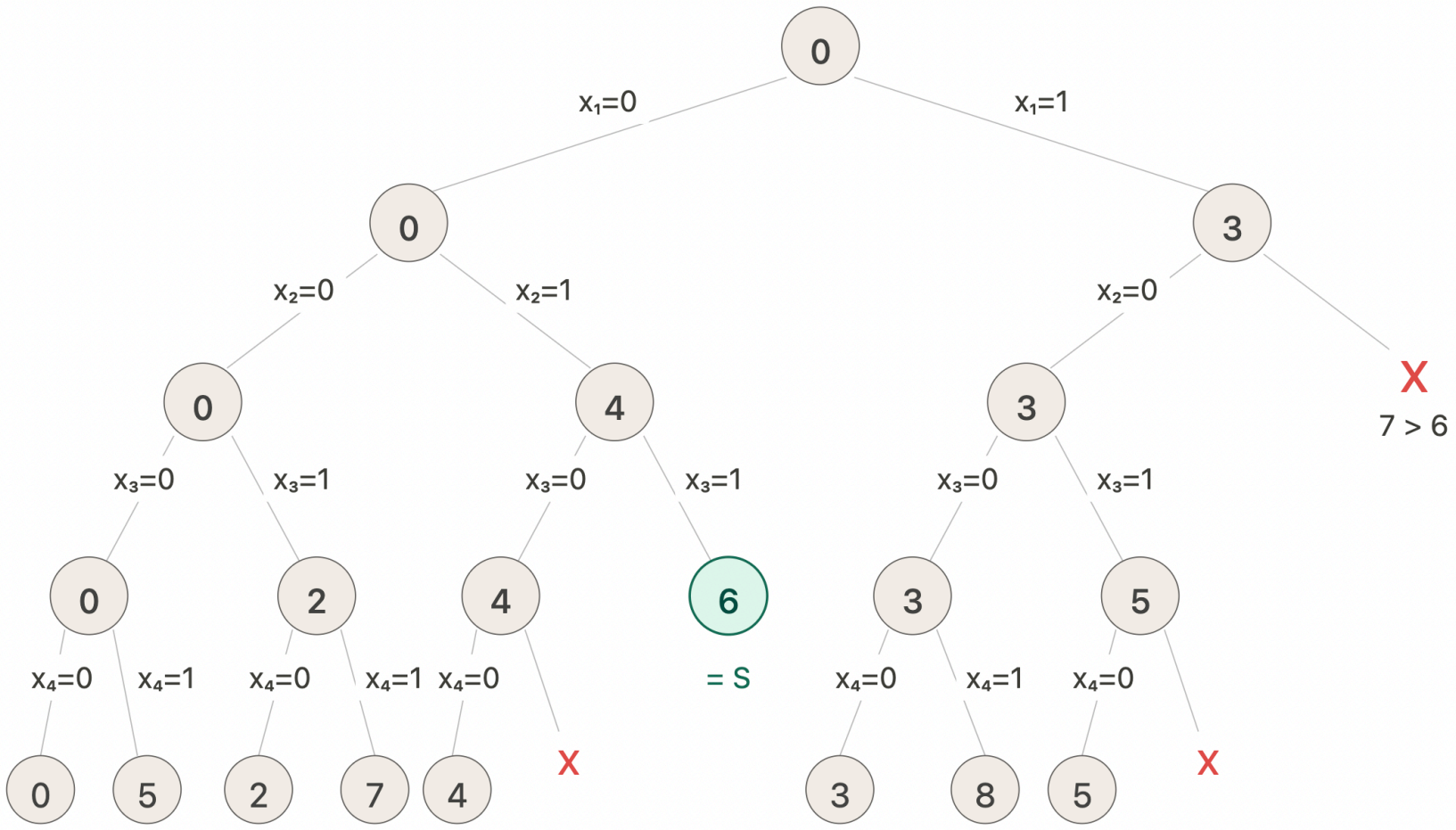


```
includido[1..n]=inicializado com 0's

def soma_conj(A, n)
    s = 0
    para i = 1 até n
        s += includido[i] * A[i]
    retornar s

def soma(A, S, n)
    achou = Verdadeiro
    enquanto achou:
    se soma_conj(A, n) == S
       imprimir(includido)
    achou = proximo(n)
```

- O código acima tem um defeito de eficiência: soma_conj recalcula a soma inteira a cada subconjunto, quando próximo só altera alguns bits
- Poderíamos manter a soma de forma incremental, ajustando apenas os elementos que mudaram. Mas para isso, próximo precisaria conhecer o vetor A e atualizar a soma e perdemos a separação entre enumeração e processamento

Se A[i] > 0 para todos os elementos, quando a soma parcial já ultrapassou S, podemos interromper a busca nesse ramo. Isso é chamado de **poda**. Vamos ver como aplicar isso na implementação recursiva:

```
def soma(A, S, i)
  se i == n + 1
      se S == 0
          imprimir(incluido)
      retornar
  incluido[i] = 0
  soma(A, S, i + 1)
  se S >= A[i]
      incluido[i] = 1
      soma(A, S - A[i], i + 1)
```



= solução X = poda

A = [3, 4, 2, 5], S = 6

Problema: Caixeiro Viajante

Entrada: Uma matrix de custos C de inteiros, onde C[i][j] é um inteiro >=0 é o custo de ir de i para j.

Saída: Uma permutação $(p_1, p_2, ..., p_n)$ com $p_1 = 1$ que minimiza

$$C[p_1][p_2] + C[p_2][p_3] + ... + C[p_{n-1}][p_n] + C[p_n][p_1]$$

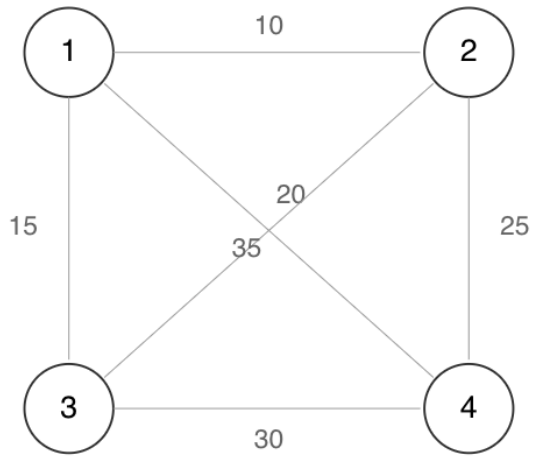
Exemplo:

Matriz de custos C:

	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0

Matriz de custos C:

	1	2	3	4
1	0	10	15	20
2	10	0	35	25
3	15	35	0	30
4	20	25	30	0



Melhor ciclo:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

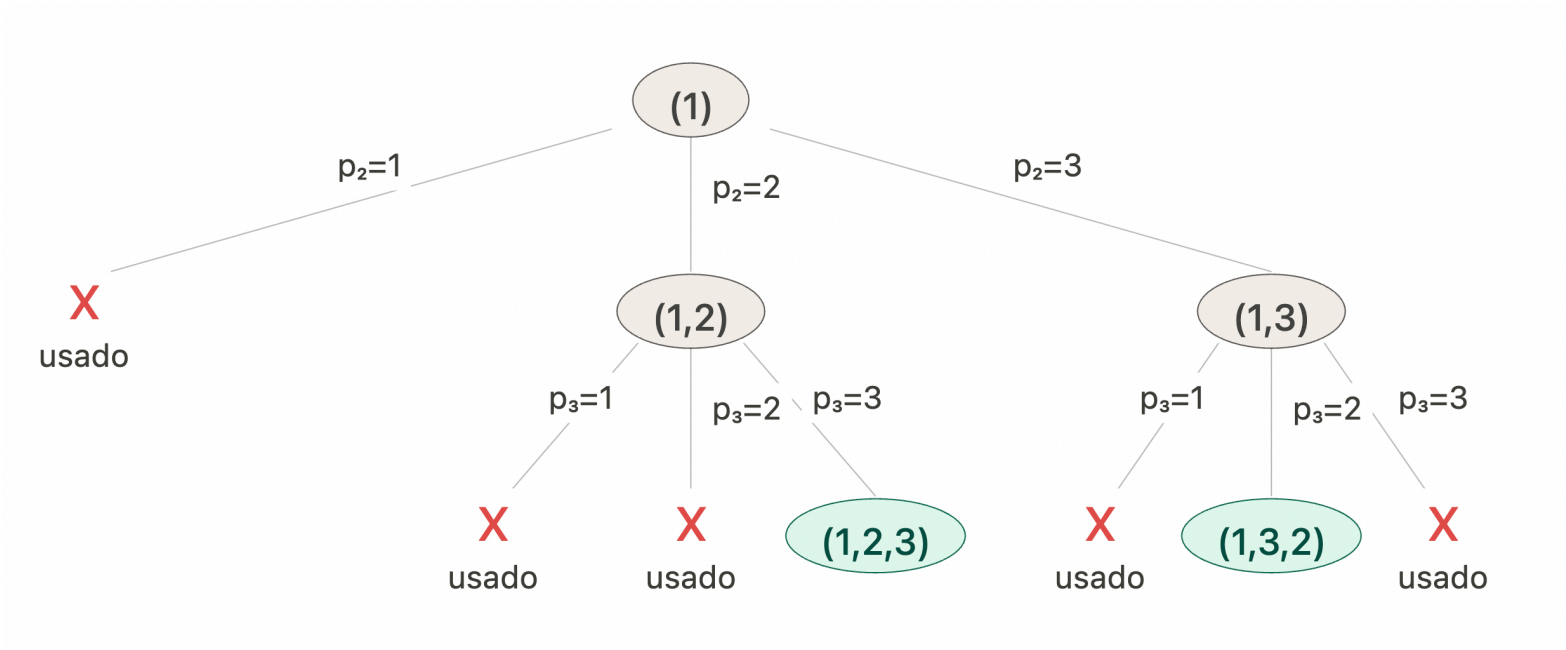
Custo: $10 + 25 + 30 + 15 = 80$

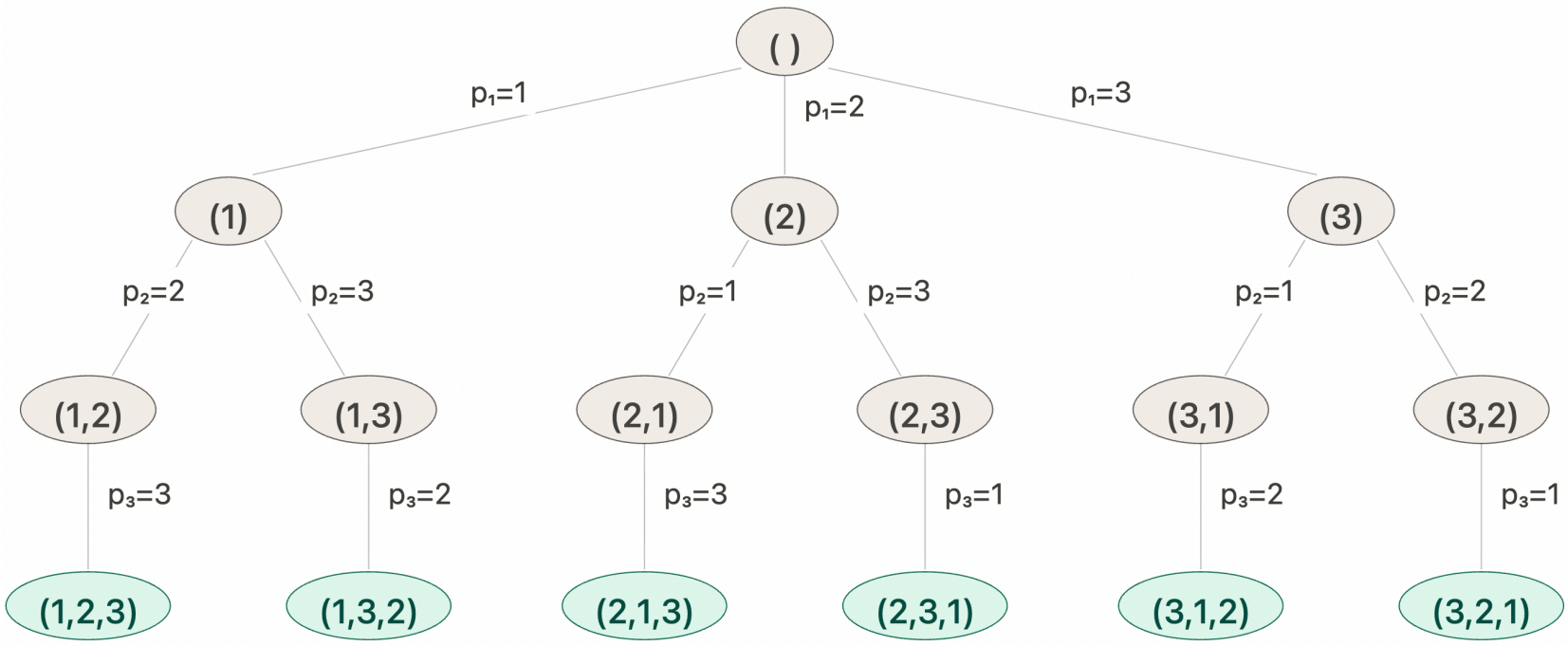
$(p_1, p_2, \ldots, p_n) = (1,2,4,3)$

- Este problema está claramente relacionado ao problema de enumerar permutações, ou seja, todas as n! possíveis ordenações de {1, 2, ..., n}
- Para este problema, a primeira coordenada deve ser 1. Mas vamos ver o caso geral primeiro.
- Para gerar permutações é importante saber quais números já foram usados para que não sejam repetidos. Vamos usar um vetor **incluido** novamente.

```
p[1..n]
incluido[1..n] = 0

def permutacoes(k)
    se k == n + 1
        imprimir p[1..n]
        retornar
    para i = 1 até n
        se incluido[i] == 0
            p[k] = i
            incluido[i] = 1
            permutacoes(k + 1)
            incluido[i] = 0
```

Note que colocamos i como p_k, geramos todas as permutações com $p_k = i$, e depois desfazemos a inclusão de i em incluido[i] = 0.


(1)
p₂=1
p₂=2
p₂=3
X
usado
(1,2)
(1,3)
p₃=1
p₃=2
p₃=3
p₃=1
p₃=2
p₃=3
X
usado
X
usado
(1,2,3)
X
usado
(1,3,2)
X
usado


( )
p₁=1
p₁=2
p₁=3
(1)
(2)
(3)
p₂=2
p₂=3
p₂=1
p₂=3
p₂=1
p₂=2
(1,2)
(1,3)
(2,1)
(2,3)
(3,1)
(3,2)
p₃=3
p₃=2
p₃=3
p₃=1
p₃=2
p₃=1
(1,2,3)
(1,3,2)
(2,1,3)
(2,3,1)
(3,1,2)
(3,2,1)

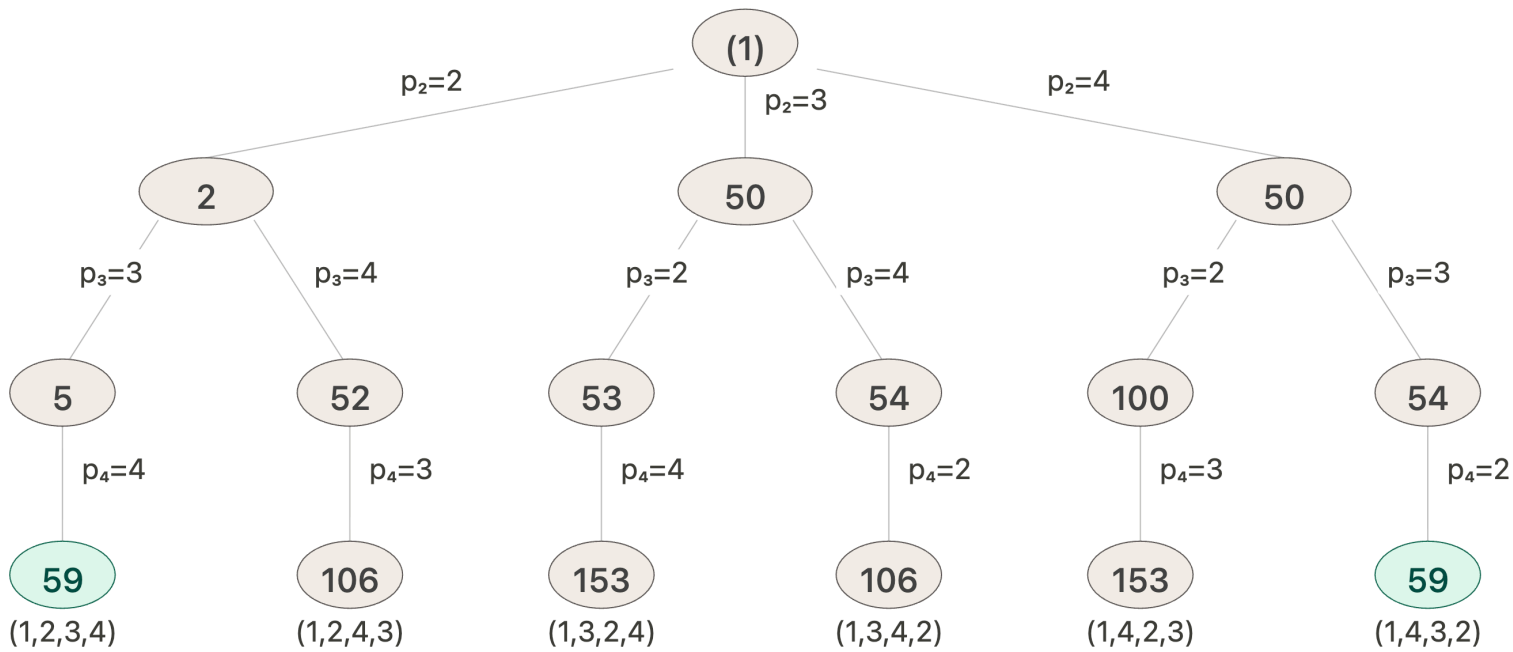
Para o problema do caixeiro viajante, temos que considerar que cada permutação tem um custo. Primeiro vamos fazer uma função que calcula o custo da permutação gerada.

```
def custo(C, n)
    s = 0
    para i = 1 até n - 1
        s += C[p[i]][p[i+1]]
    s += C[p[n]][p[1]]
    retornar s
```

Assim, podemos resolver o problema comparando o valor de cada permutação com o melhor obtido até o momento:

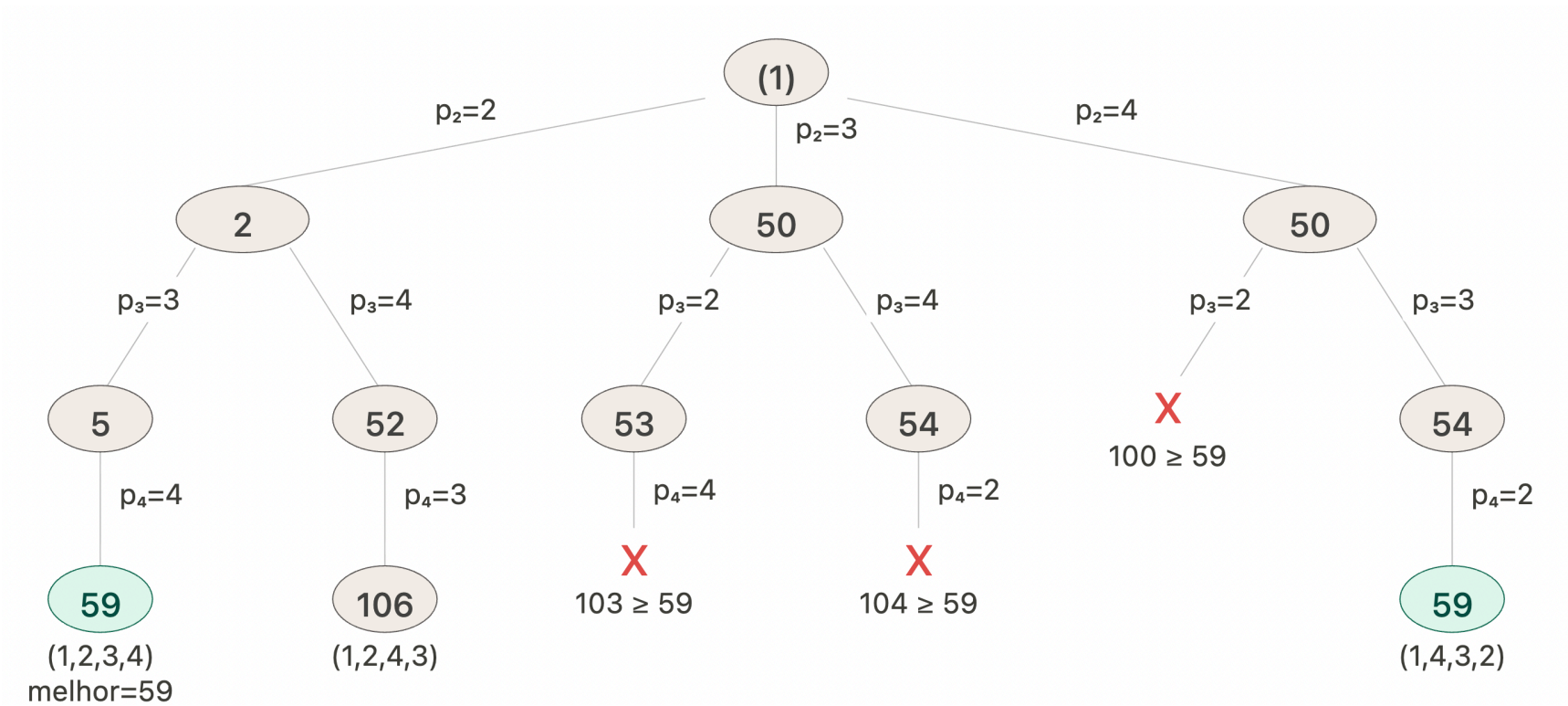
```
p[1..n] com p[1]=1
incluido[1..n] = [1,0,...,0]
melhor = infinito

def caixeiro(C, n, k)
  se k == n + 1
    c = custo(C, n)
    se c < melhor
      melhor = c
senão:
  para i = 2 até n
    se incluido[i] == 0
      p[k] = i
      incluido[i] = 1
      caixeiro(C,n,k+1)
      incluido[i] = 0
```

$$
C = \begin{array}{c|cccc}
 & 1 & 2 & 3 & 4 \\
\hline
1 & 0 & 2 & 50 & 50 \\
2 & 2 & 0 & 3 & 50 \\
3 & 50 & 3 & 0 & 4 \\
4 & 50 & 50 & 4 & 0
\end{array}
$$



- Aqui temos uma poda clara a ser feita também. Como $C[i][j] \geq 0$, se o custo da sub-rota atual já ultrapassou melhor, podemos podar esse ramo.
- Nesse caso, precisamos manter um custo parcial durante a busca.

$$
C = \begin{array}{c|cccc}
 & 1 & 2 & 3 & 4 \\
\hline
1 & 0 & 2 & 50 & 50 \\
2 & 2 & 0 & 3 & 50 \\
3 & 50 & 3 & 0 & 4 \\
4 & 50 & 50 & 4 & 0
\end{array}
$$



```
def caixeiro(C, n, k, parcial)
  se k == n + 1
    se parcial+C[p[n]][p[1]] < melhor
      melhor = parcial+C[p[n]][p[1]]
  senão
    para i = 2 até n
      se incluido[i] == 0
        p[k] = i
        incluido[i] = 1
        novo = parcial + C[p[k-1]][i]
        se novo<melhor:
            caixeiro(C, n, k + 1,novo)
        incluido[i] = 0
```